HIGH PERFORMANCE COMPUTING

ASSIGNMENT-11

Name : A.JEEVAN SAI

HT No : 2303A51420

Batch No : 21

Assignment 1: Distributed Temperature Monitoring System

Concept: Worker processes act as sensors generating temperature data, sending it to the Master process using blocking communication (MPI_Send / MPI_Recv). The Master computes the maximum and average temperatures.

Python Code (assignment_1.py)

```
from mpi4py import MPI
import random

def main():
  comm = MPI.COMM_WORLD
  rank = comm.Get_rank()
  size = comm.Get_size()

  # Ensure we have at least 1 master and 1 worker
  if size < 2:
    if rank == 0: print("Please run with at least 2 processes (e.g., mpiexec -n 4 python
      assignment_1.py)")
    return

  if rank == 0: #
    Master Process
    temperatures = []
    print(f"Master (Process {rank}) waiting for sensor data...")

    # Receive temperature from each worker sequentially
    for i in range(1, size):
      # Blocking receive from worker 'i'
      temp = comm.recv(source=i, tag=1)
      temperatures.append(temp)
      print(f"Master received temperature {temp:.2f}°C from Node {i}")

    # Compute metrics
```

```
        avg_temp = sum(temperatures) / len(temperatures)
        max_temp = max(temperatures)

        print(f"\n--- Temperature Report ---")
        print(f"Average Temperature: {avg_temp:.2f}°C")
        print(f"Maximum Temperature: {max_temp:.2f}°C")
        print("--------------------------")

    else:
        # Worker Process (Sensor Node)
        # Generate random temperature between 20.0 and 40.0
        temp = random.uniform(20.0, 40.0)
        print(f"Node {rank} generated temperature {temp:.2f}°C")

        # Blocking send to Master (Process 0)
        comm.send(temp, dest=0, tag=1)

if __name__ == "__main__":
    main()
```

Output

(Running with 4 processes: mpiexec -n 4 python assignment_1.py)

```
Node 1 generated temperature 23.45°C
Node 2 generated temperature 38.12°C
Node 3 generated temperature 29.80°C
Master (Process 0) waiting for sensor data...
Master received temperature 23.45°C from Node 1
Master received temperature 38.12°C from Node 2
Master received temperature 29.80°C from Node 3

--- Temperature Report --Average
Temperature: 30.46°C
Maximum Temperature: 38.12°C
--------------------------
```

Assignment 2: Parallel Image Row Processing

Concept: The Master holds an 8x8 matrix representing an image. It distributes the rows among worker processes. The workers apply a simple filter (increasing brightness) and return the rows back to the Master to reconstruct the image.

Python Code (assignment_2.py)

```
from mpi4py import MPI

def main():
  comm = MPI.COMM_WORLD
  rank = comm.Get_rank()
  size = comm.Get_size()

  BRIGHTNESS_INCREASE = 50
  ROWS = 8 COLS
  = 8

  if size < 2:
    if rank == 0:
      print("Please run with at least 2 processes.")
    return

  if rank == 0:
    # Master Process: Create a mock 8x8 grayscale image (pixel values 0-255)
    image = [[r * 10 + c for c in range(COLS)] for r in range(ROWS)]
    print(f"Master (Process 0): Distributing image rows...") print("Original
    Row 0:", image[0]) print("Original Row 1:", image[1]) num_workers = size
    - 1

    # Phase 1: Distribute rows to workers (Round-Robin)
    for r in range(ROWS):
      worker_dest = (r % num_workers) + 1
      # Send a tuple containing the row index and the row data itself
      comm.send((r, image[r]), dest=worker_dest, tag=11)

    # Send termination signal (-1) to all workers so they know to
    stop for w in range(1, size): comm.send((-1, []), dest=w, tag=11)
```

```
    # Phase 2: Receive processed rows back from workers
    processed_image = [[] for _ in range(ROWS)] for r in
    range(ROWS):
      worker_source = (r % num_workers) + 1
      row_idx, processed_row = comm.recv(source=worker_source, tag=22)
      processed_image[row_idx] = processed_row

    print(f"\nMaster (Process 0): Image reconstructed successfully.")
    print("Processed Row 0:", processed_image[0]) print("Processed
    Row 1:", processed_image[1])

  else:
    # Worker Process
    while True:
      # Receive row from Master
      row_idx, row = comm.recv(source=0, tag=11)

      # Check for termination
      signal if row_idx == -1: break

      # Increase brightness, making sure not to exceed max pixel value (255)
      processed_row = [min(255, pixel + BRIGHTNESS_INCREASE) for pixel in row]

      # Send the processed row back to Master
      comm.send((row_idx, processed_row), dest=0, tag=22)

if __name__ == "__main__":
  main()
```

Output

(Running with 4 processes: mpiexec -n 4 python assignment_2.py)

```
Master (Process 0): Distributing image rows...
Original Row 0: [0, 1, 2, 3, 4, 5, 6, 7]
Original Row 1: [10, 11, 12, 13, 14, 15, 16, 17]

Master (Process 0): Image reconstructed successfully.
Processed Row 0: [50, 51, 52, 53, 54, 55, 56, 57]
Processed Row 1: [60, 61, 62, 63, 64, 65, 66, 67]
```

Assignment 3: Ring Communication in Network Simulation

Concept: Processes are arranged in a logical ring. A token starts at Process 0 and travels entirely around the ring. Every process receives the token, adds its own rank to the token's value, and forwards it to the next process until it returns to Process 0.

Python Code (assignment_3.py)

```
from mpi4py import MPI

def main():
  comm = MPI.COMM_WORLD
  rank = comm.Get_rank()
  size = comm.Get_size()

  if size < 2:
    if rank == 0: print("Please run with at least 2
      processes.")
    return

  if rank == 0:
    # Process 0 starts the token passing
    token = 1
    print(f"Process {rank} starting token with value {token}")

    # Send to the next process (Process 1)
    print(f"Process {rank} sending token {token} to Process 1")
    comm.send(token, dest=1, tag=33)

    # Wait to receive the final token from the last process
    final_token = comm.recv(source=size-1, tag=33)
    print(f"Process {rank} received FINAL token back with value: {final_token}")

  else:
    # Intermediate Processes
    # 1. Receive token from previous process
    prev_process = rank - 1
    token = comm.recv(source=prev_process, tag=33)
    print(f"Process {rank} received token {token} from Process {prev_process}")

    # 2. Add current rank to token
    token += rank
```

```
    # 3. Send token to next process (wrap around to 0 if it's the last process)
    next_process = (rank + 1) % size
    print(f"Process {rank} sending token {token} to Process {next_process}")
    comm.send(token, dest=next_process, tag=33)

if __name__ == "__main__":
  main()
```

Output

(Running with 4 processes: mpiexec -n 4 python assignment_3.py)

```
Process 0 starting token with value 1
Process 0 sending token 1 to Process 1
Process 1 received token 1 from Process 0
Process 1 sending token 2 to Process 2
Process 2 received token 2 from Process 1
Process 2 sending token 4 to Process 3
Process 3 received token 4 from Process 2
Process 3 sending token 7 to Process 0
Process 0 received FINAL token back with value: 7
```